

```
In [1]: import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]   C:\Users\CNLAB07\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]   C:\Users\CNLAB07\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]   C:\Users\CNLAB07\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   C:\Users\CNLAB07\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]   date!
```

```
Out[1]: True
```

```
In [3]: text = "Tokenization is the first step in text analytics. The process of breaking down a text paragraph into smaller
```

```
In [5]: # Sentence Tokenization
from nltk.tokenize import sent_tokenize
tokenized_text = sent_tokenize(text)
print("Sentence Tokenization:", tokenized_text)

# Word Tokenization
from nltk.tokenize import word_tokenize
tokenized_word = word_tokenize(text)
print("Word Tokenization:", tokenized_word)
```

```
Sentence Tokenization: ['Tokenization is the first step in text analytics.', 'The process of breaking down a text paragraph into smaller chunks such as words or sentences is called Tokenization.']
Word Tokenization: ['Tokenization', 'is', 'the', 'first', 'step', 'in', 'text', 'analytics', '.', 'The', 'process', 'of', 'breaking', 'down', 'a', 'text', 'paragraph', 'into', 'smaller', 'chunks', 'such', 'as', 'words', 'or', 'sentences', 'is', 'called', 'Tokenization', '.']
```

```
In [7]: from nltk.corpus import stopwords
import re
```

```
stop_words = set(stopwords.words("english"))
print("Stop Words:", stop_words)

text = "How to remove stop words with NLTK library in Python?"
text = re.sub('[^a-zA-Z]', ' ', text) # Remove non-alphabetic characters
tokens = word_tokenize(text.lower())
filtered_text = [w for w in tokens if w not in stop_words]

print("Tokenized Sentence:", tokens)
print("Filtered Sentence:", filtered_text)
```

Stop Words: {'against', 'shan', "you'd", 'we', 'ma', 'themselves', 'up', 'was', 'did', 'mightn', 'his', 'and', "it'll", 's', "won't", 'won', 'your', 'very', 'only', 'the', 'own', 'haven', 'who', 'yourself', 'doesn', 'but', "she'd", "we'd", 'doing', 'does', 'aren', 'had', 'hadn', "you're", 'same', 'couldn', 'off', 'than', "you've", "hasn't", 'out', 'such', 'he', 'with', 'again', 'at', 'o', 'myself', "we've", 'theirs', 'needn', 'between', 'each', "weren't", "needn't", 'further', "he'll", 'other', 'as', "isn't", 'do', 'about', 'not', 'why', "wasn't", 'been', 'nor', 'for', 'if', 'itself', 'him', 'don', 'an', 'over', 'any', 'some', "hadn't", 'how', "she's", 'by', 'is', 'those', 't', 'should', 'w hile', 'both', 'now', 'her', "you'll", 'where', "they've", 'below', 'or', 'these', 'shouldn', "should've", "we'll", "they'll", 'because', 'isn', 'most', 'down', 'there', "i'm", 'on', 'of', "i've", "that'll", 'here', "we're", 'its', "he'd", 've', 'has', "it's", "shouldn't", 'ours', 'that', 'before', 'were', 'having', "mightn't", 'yourselves', "they'd", 'd', 'yours', "couldn't", 'what', 'into', 'once', 'y', 'they', 'mustn', "it'd", 'being', 'am', "don't", 'too', "doesn't", 'then', 'after', 'so', 're', 'himself', 'me', 'our', 'which', 'she', 'm', 'have', 'are', 'weren', 'herself', 'hers', "he's", 'just', 'no', 'through', 'my', 'to', 'wouldn', 'whom', "she'll", "they're", 'ourselves', 'i', 'can', 'all', 'this', "aren't", 'their', 'from', 'it', 'you', 'be', 'a', "haven't", 'until', 'when', "didn't", 'hasn', "shan't", 'wasn', "wouldn't", 'in', "i'd", 'ain', 'will', 'few', 'didn', 'during', 'above', "i'll", 'them', 'under', 'll', "mustn't", 'more'}

Tokenized Sentence: ['how', 'to', 'remove', 'stop', 'words', 'with', 'nltk', 'library', 'in', 'python']

Filtered Sentence: ['remove', 'stop', 'words', 'nltk', 'library', 'python']

```
In [9]: from nltk.stem import PorterStemmer
```

```
e_words = ["wait", "waiting", "waited", "waits"]
ps = PorterStemmer()

for w in e_words:
    rootWord = ps.stem(w)
    print(f"Stemmed Word for '{w}': {rootWord}")
```

Stemmed Word for 'wait': wait
Stemmed Word for 'waiting': wait
Stemmed Word for 'waited': wait
Stemmed Word for 'waits': wait

```
In [11]: from nltk.stem import WordNetLemmatizer
wordnet_lemmatizer = WordNetLemmatizer()

text = "studies studying cries cry"
tokenization = nltk.word_tokenize(text)

for w in tokenization:
    print(f"Lemma for {w}: {wordnet_lemmatizer.lemmatize(w)}")
```

Lemma for studies: study
Lemma for studying: studying
Lemma for cries: cry
Lemma for cry: cry

```
In [13]: data = "The pink sweater fit her perfectly"
words = word_tokenize(data)

for word in words:
    print(nltk.pos_tag([word]))
```

[('The', 'DT')]
[('pink', 'NN')]
[('sweater', 'NN')]
[('fit', 'NN')]
[('her', 'PRP\$')]
[('perfectly', 'RB')]

```
In [15]: import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
import math
```

```
In [17]: documentA = 'Jupiter is the largest Planet'
documentB = 'Mars is the fourth planet from the Sun'
```

```
In [19]: bagOfWordsA = documentA.split(' ')
bagOfWordsB = documentB.split(' ')
```

```
In [21]: uniqueWords = set(bagOfWordsA).union(set(bagOfWordsB))
```

```
In [23]: numOfWordsA = dict.fromkeys(uniqueWords, 0)
        for word in bagOfWordsA:
            numOfWordsA[word] += 1

        numOfWordsB = dict.fromkeys(uniqueWords, 0)
        for word in bagOfWordsB:
            numOfWordsB[word] += 1
```

```
In [25]: def computeTF(wordDict, bagOfWords):
        tfDict = {}
        bagOfWordsCount = len(bagOfWords)
        for word, count in wordDict.items():
            tfDict[word] = count / float(bagOfWordsCount)
        return tfDict

        tfA = computeTF(numOfWordsA, bagOfWordsA)
        tfB = computeTF(numOfWordsB, bagOfWordsB)

        print("Term Frequency for Document A:", tfA)
        print("Term Frequency for Document B:", tfB)
```

Term Frequency for Document A: {'Planet': 0.2, 'planet': 0.0, 'largest': 0.2, 'from': 0.0, 'Jupiter': 0.2, 'fourth': 0.0, 'is': 0.2, 'Sun': 0.0, 'Mars': 0.0, 'the': 0.2}

Term Frequency for Document B: {'Planet': 0.0, 'planet': 0.125, 'largest': 0.0, 'from': 0.125, 'Jupiter': 0.0, 'fourth': 0.125, 'is': 0.125, 'Sun': 0.125, 'Mars': 0.125, 'the': 0.25}

```
In [27]: def computeIDF(documents):
        N = len(documents)
        idfDict = dict.fromkeys(documents[0].keys(), 0)
        for document in documents:
            for word, val in document.items():
                if val > 0:
                    idfDict[word] += 1
        for word, val in idfDict.items():
            idfDict[word] = math.log(N / float(val))
        return idfDict

        idfs = computeIDF([numOfWordsA, numOfWordsB])
        print("Inverse Document Frequency:", idfs)
```

Inverse Document Frequency: {'Planet': 0.6931471805599453, 'planet': 0.6931471805599453, 'largest': 0.6931471805599453, 'from': 0.6931471805599453, 'Jupiter': 0.6931471805599453, 'fourth': 0.6931471805599453, 'is': 0.0, 'Sun': 0.6931471805599453, 'Mars': 0.6931471805599453, 'the': 0.0}

```
In [29]: def computeTFIDF(tfBagOfWords, idfs):
        tfidf = {}
        for word, val in tfBagOfWords.items():
            tfidf[word] = val * idfs[word]
        return tfidf

        tfidfA = computeTFIDF(tfA, idfs)
        tfidfB = computeTFIDF(tfB, idfs)

        print("TF-IDF for Document A:", tfidfA)
        print("TF-IDF for Document B:", tfidfB)
```

TF-IDF for Document A: {'Planet': 0.13862943611198905, 'planet': 0.0, 'largest': 0.13862943611198905, 'from': 0.0, 'Jupiter': 0.13862943611198905, 'fourth': 0.0, 'is': 0.0, 'Sun': 0.0, 'Mars': 0.0, 'the': 0.0}

TF-IDF for Document B: {'Planet': 0.0, 'planet': 0.08664339756999316, 'largest': 0.0, 'from': 0.08664339756999316, 'Jupiter': 0.0, 'fourth': 0.08664339756999316, 'is': 0.0, 'Sun': 0.08664339756999316, 'Mars': 0.08664339756999316, 'the': 0.0}

```
In [31]: df = pd.DataFrame([tfidfA, tfidfB], index=["Document A", "Document B"])
        print("TF-IDF DataFrame:\n", df)
```

TF-IDF DataFrame:

	Planet	planet	largest	from	Jupiter	fourth	is	\
Document A	0.138629	0.000000	0.138629	0.000000	0.138629	0.000000	0.0	
Document B	0.000000	0.086643	0.000000	0.086643	0.000000	0.086643	0.0	

	Sun	Mars	the
Document A	0.000000	0.000000	0.0
Document B	0.086643	0.086643	0.0